README

motor_ws — 갠트리 로봇 모터 제어 (듀얼 갠트리)

ROS2 Humble 기반 EtherCAT 듀얼 갠트리 모터 제어 패키지.
단일 EtherCAT 체인에 iX7NH 서보 드라이버 **6대** (갠트리0·갠트리1 각 3축) 를 연결하여
단일 노드로 두 갠트리를 동시에 제어합니다.

목차

1. 시스템 구성
2. 소프트웨어 요구사항
3. 빌드
4. 설정 파일
5. 실행
6. 호밍
7. 이동 명령
8. 테스트 모드
9. ROS2 인터페이스
10. 문서

---

1. 시스템 구성

```
[Jetson (Ubuntu 22.04)]
        │  EtherCAT (enP8p1s0)
        ├─ Slave 0 : iX7NH → 갠트리0 X축  (NOT 리밋)
        ├─ Slave 1 : iX7NH → 갠트리0 Z1축 (POT 리밋)
        ├─ Slave 2 : iX7NH → 갠트리0 Z2축 (POT 리밋)
        ├─ Slave 3 : iX7NH → 갠트리1 X축  (NOT 리밋)
        ├─ Slave 4 : iX7NH → 갠트리1 Z1축 (POT 리밋)
        └─ Slave 5 : iX7NH → 갠트리1 Z2축 (POT 리밋)
```

항목	내용
OS	Ubuntu 22.04 (NVIDIA Jetson)
ROS2	Humble
EtherCAT 라이브러리	pysoem
드라이브 프로파일	CiA 402
운전 모드	CSV (Cyclic Synchronous Velocity, Mode 9)
호밍 모드	CiA 402 Homing Mode (Mode 6)

항목	내용
엔코더 분해능	8,388,608 pulse/rev (2^{23})
슬레이브 수	6대 (갠트리당 X 1대 + Z 2대)

---

2. 소프트웨어 요구사항

```
# ROS2 Humble
sudo apt install ros-humble-desktop

# pysoem (반드시 root 환경에 설치)
sudo pip3 install pysoem

# ROS2 의존성
sudo apt install python3-argcomplete python3-pip
```

> **중요:** EtherCAT은 `CAP_NET_RAW` 권한이 필요합니다.
> pysoem 서브프로세스에서는 파일 capability가 동작하지 않으므로 **반드시 `sudo -s` (root)** 로 실행해야 합니다.

---

3. 빌드

```
cd ~/motor_ws

source /opt/ros/humble/setup.bash
colcon build --symlink-install

# ldconfig 업데이트 (최초 1회)
echo "/opt/ros/humble/lib" | sudo tee /etc/ld.so.conf.d/ros-humble.conf
echo "$(pwd)/install/motor_control_interfaces/lib" | \
  sudo tee /etc/ld.so.conf.d/motor-ws.conf
sudo ldconfig
```

---

4. 설정 파일 (motor_params.yaml)

경로: `src/motor_control/config/motor_params.yaml`

슬레이브 인덱스

```
num_slaves: 6

# 갠트리 0
gantry0_x_slave_idx:  0
```

```yaml
gantry0_z1_slave_idx: 1
gantry0_z2_slave_idx: 2

# 갠트리 1
gantry1_x_slave_idx:  3
gantry1_z1_slave_idx: 4
gantry1_z2_slave_idx: 5
```

사용하지 않는 축은 `-1` 로 설정하면 비활성화됩니다.

주요 파라미터

파라미터	기본값	설명
`ifname`	`enP8p1s0`	EtherCAT 네트워크 인터페이스
`num_slaves`	`6`	총 EtherCAT 슬레이브 수
`pdo_cycle_ms`	`10`	EtherCAT PDO 사이클 (ms)
`state_publish_hz`	`10.0`	AxisState 토픽 발행 주기 (Hz)
`default_velocity_rpm`	`30.0`	기본 이동 속도 (RPM)
`default_velocity_mm_s`	`75.0`	기본 이동 속도 (mm/s)
`default_accel_rpm_s`	`1000.0`	기본 가속도 (RPM/s)
`homing_fast_rpm`	`30.0`	호밍 탐색 속도 (RPM)
`homing_slow_rpm`	`1.0`	호밍 Index 탐색 속도 (RPM)
`homing_timeout_s`	`1200.0`	호밍 타임아웃 (초, 두 갠트리 합산)
`max_sync_error_mm`	`10.0`	Z1-Z2 동기화 오차 긴급정지 임계값 (mm)
`gantry0_x_soft_limit_max_mm`	`1750.0`	갠트리0 X 소프트 리밋 최댓값 (mm)
`gantry0_z_soft_limit_min_mm`	`-1750.0`	갠트리0 Z 소프트 리밋 최솟값 (mm)
`gantry1_x_soft_limit_max_mm`	`1750.0`	갠트리1 X 소프트 리밋 최댓값 (mm)
`gantry1_z_soft_limit_min_mm`	`-1750.0`	갠트리1 Z 소프트 리밋 최솟값 (mm)

---

5. 실행

```bash
# 반드시 root로 실행
sudo -s

source /opt/ros/humble/setup.bash
source ~/motor_ws/install/setup.bash

ros2 launch motor_control motor_control.launch.py
```

정상 시작 로그

```
[EtherCAT] 6개 슬레이브 발견
  Slave 0 (X): 호밍 모드(6) 사전 설정 완료 (method=1)
  ...
  Slave 5 (Z): 호밍 모드(6) 사전 설정 완료 (method=2)
[EtherCAT] OP 상태 진입 완료
motor_driver_node 준비 완료.
```

---

6. 호밍

`gantry_index` 로 대상 갠트리를 지정합니다.

gantry_index	동작
`0`	갠트리0만 호밍
`1`	갠트리1만 호밍
`2`	두 갠트리 **동시** 호밍

```
# 두 갠트리 동시 호밍
ros2 action send_goal /motor/homing motor_control_interfaces/action/Homing \
  "{gantry_index: 2, force_rehome: true}"

# 갠트리0만 호밍
ros2 action send_goal /motor/homing motor_control_interfaces/action/Homing \
  "{gantry_index: 0, force_rehome: false}"
```

---

7. 이동 명령

`gantry_index` 로 대상 갠트리를 지정하고, 이동하지 않을 축에는 `-1.0e9` 를 지정합니다.

gantry_index	동작	사용 파라미터
`0`	갠트리0만 이동	`gantry0_x_target_mm`, `gantry0_z_target_mm`
`1`	갠트리1만 이동	`gantry1_x_target_mm`, `gantry1_z_target_mm`
`2`	두 갠트리 **동시·동일** 이동	`gantry2_x_target_mm`, `gantry2_z_target_mm`

```
# 두 갠트리 동시·동일 이동 (X=100mm, Z=-50mm)
ros2 action send_goal /motor/move motor_control_interfaces/action/MoveAxis "{
  gantry_index: 2,
  gantry0_x_target_mm: -1.0e9, gantry0_z_target_mm: -1.0e9,
  gantry1_x_target_mm: -1.0e9, gantry1_z_target_mm: -1.0e9,
  gantry2_x_target_mm: 100.0,  gantry2_z_target_mm: -50.0,
  velocity_mm_s: 30.0, force_move: false}"
```

```
# 갠트리0만 이동 (X=200mm, Z 유지)
ros2 action send_goal /motor/move motor_control_interfaces/action/MoveAxis "{
  gantry_index: 0,
  gantry0_x_target_mm: 200.0,  gantry0_z_target_mm: -1.0e9,
  gantry1_x_target_mm: -1.0e9, gantry1_z_target_mm: -1.0e9,
  gantry2_x_target_mm: -1.0e9, gantry2_z_target_mm: -1.0e9,
  velocity_mm_s: 50.0, force_move: false}"
```

자세한 이동 명령 예시: [DUAL_GANTRY_GUIDE.md]

---

8. 테스트 모드

사용하지 않는 축의 슬레이브 인덱스를 `-1` 로 설정하고 `num_slaves` 를 실제 연결 대수로 조정합니다.

모드	num_slaves	g0_x	g0_z1	g0_z2	g1_x	g1_z1	g1_z2
갠트리0 X 단독	1	0	-1	-1	-1	-1	-1
갠트리0 전체	3	0	1	2	-1	-1	-1
갠트리1 전체	3	-1	-1	-1	0	1	2
듀얼 갠트리 (전체)	6	0	1	2	3	4	5

---

9. ROS2 인터페이스

토픽

토픽	타입	설명
`/motor/gantry0/axis_state`	`AxisState`	갠트리0 위치·속도·호밍 상태 (10 Hz)
`/motor/gantry1/axis_state`	`AxisState`	갠트리1 위치·속도·호밍 상태 (10 Hz)

액션

액션	타입	설명
`/motor/homing`	`Homing`	호밍 실행 (gantry_index: 0/1/2)
`/motor/move`	`MoveAxis`	목표 위치 이동 (gantry_index: 0/1/2)

서비스

서비스	타입	설명
`/motor/set_velocity`	`SetVelocity`	두 갠트리 공통 속도 변경

서비스	타입	설명
`/motor/get_status`	`GetDriveStatus`	드라이브 상태 조회 (gantry_index 지정)

상태 확인 명령

```
# 갠트리별 현재 위치 확인
ros2 topic echo /motor/gantry0/axis_state --once
ros2 topic echo /motor/gantry1/axis_state --once

# 갠트리0 드라이브 상태 확인
ros2 service call /motor/get_status motor_control_interfaces/srv/GetDriveStatus \
  "{gantry_index: 0}"

# 속도 변경 (두 갠트리 공통)
ros2 service call /motor/set_velocity motor_control_interfaces/srv/SetVelocity \
  "{velocity_mm_s: 30.0, axis: 'all'}"
```

---

10. 문서

파일	내용
DUAL_GANTRY_GUIDE.md	듀얼 갠트리 사용 가이드 (인터페이스·명령 예시 전체)
MOTOR_WS_GUIDE.md	시스템 아키텍처·코드 구조·API 레퍼런스
VELOCITY_GUIDE.md	속도·가속도 단위 변환
TEST_X_AXIS.md	X축 단독 테스트 절차
TEST_Z_AXIS.md	Z축 단독 테스트 절차